



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет: «Информатика и системы управления» (ИУ)

Кафедра: «Информационная безопасность» (ИУ8)

Домашняя работа № 4

по курсу «Математические основы искусственного
интеллекта»

Выполнил: Земляков А.А.
Группа: ИУ8-13М
Вариант: 3

Проверил: Каганов Ю.Т.

Москва, 2025

1. Цель работы

Разработка многослойного персептрона на основе обратного распространения ошибки FFNN.

2. Постановка задачи

2.1. Реализовать на языке высокого уровня многослойный персептрон и проверить его работоспособность на примере данных, выбранных из MNIST dataset.

2.2. Исследовать работу персептрона на основе использования различных целевых функций. (среднеквадратичная ошибка, перекрестная энтропия, дивергенция Кульбака-Лейблера).

2.3. Исследовать работу многослойного персептрона с использованием различных методов оптимизации (градиентный, Флетчера-Ривза (FR), Бroyдена-Флетчера-Гольдфарба-Шенно (BFGS)).

2.4. Провести исследование эффективности работы многослойного персептрона при изменении гиперпараметров (количества нейронов и количества слоев).

2.5. Подготовить отчет с распечаткой текста программы, графиками результатов исследования и анализом результатов.

3. Практическая часть

Для реализации данных алгоритмов была написана программа, которая приведена в Листинге А.

В результате работы программы были получены графики зависимостей, а также нейронная сеть обучилась и смогла предсказать все числа.

На рисунке 1 изображены примеры из MNIST

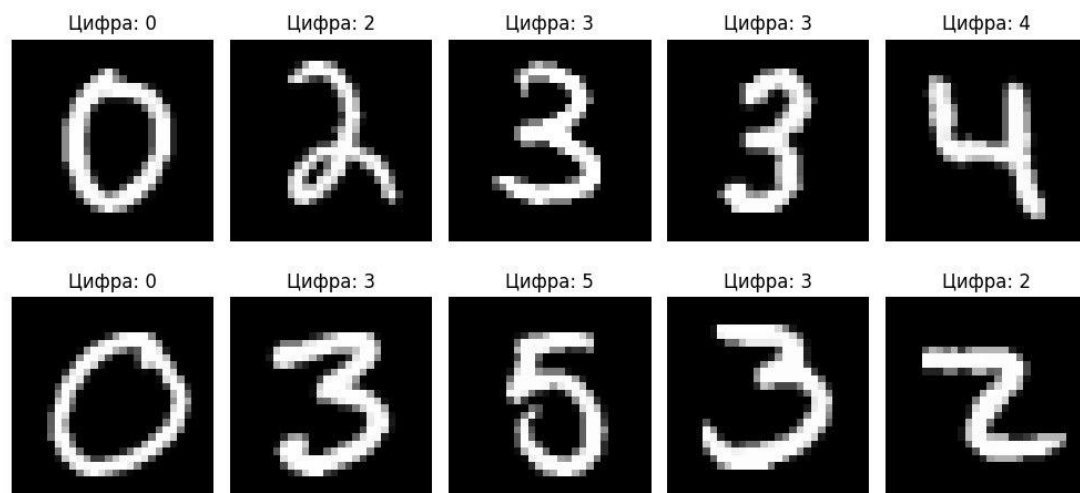


Рисунок 1 - Примеры из MNIST

На рисунке 2 изображен график, который сравнивает точность и потери в зависимости от эпохи и различных функций потерь. Наиболее точной на этапе валидации оказалась $KLDivergence$. Среднее время обучения на одной эпохе составляло порядка 9 секунд.

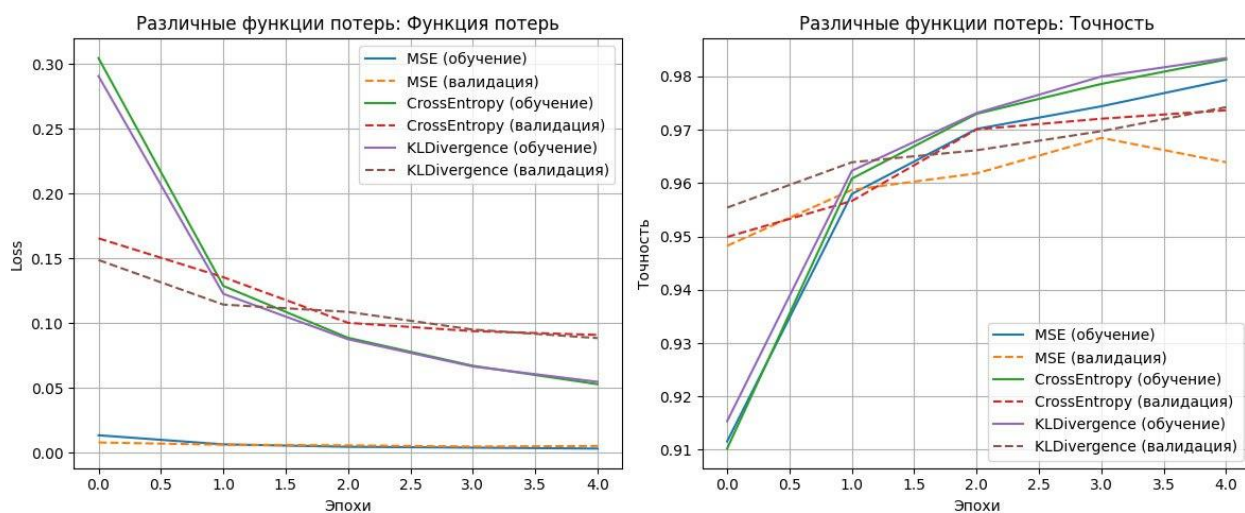


Рисунок 2 - Графики сравнений различных функций потерь

На рисунке 3 изображен результат работы нейросети - предсказанное значение и реальное, а также фотография, которая подавалась на вход. Все предсказания совпали с истинными значениями.

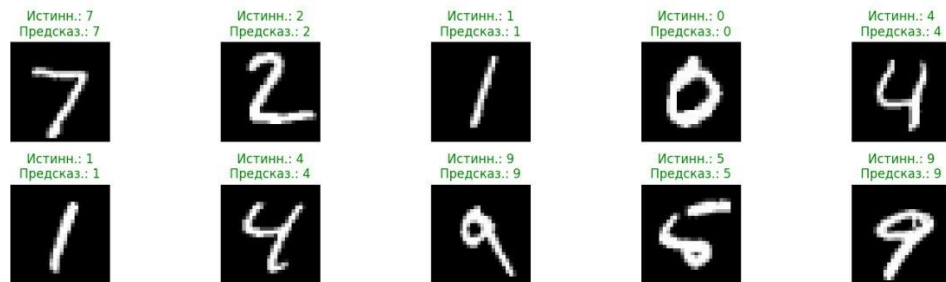


Рисунок 3 - Результат работы

На рисунке 4 изображено распределение весов первого слоя, то есть - какое значение встречается наиболее часто

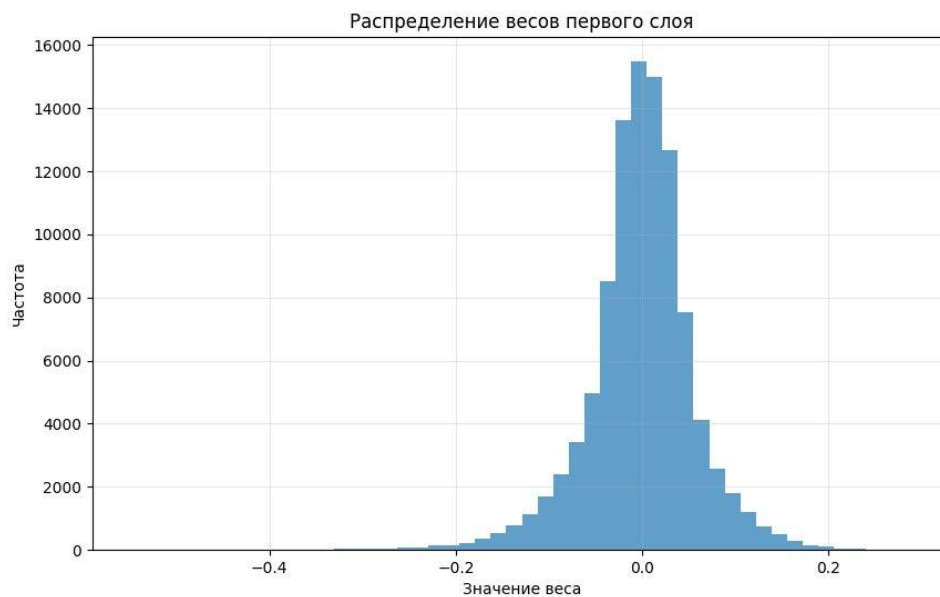


Рисунок 4 - Распределение весов первого слоя

Листинг А

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import time
import os
from torch.utils.data import DataLoader, Subset

# Установка случайных чисел для воспроизводимости результатов
torch.manual_seed(42)
np.random.seed(42)

# Проверка доступности GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# 1. Подготовка данных MNIST
print("Загрузка и подготовка данных MNIST...")
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)) # Стандартные значения для MNIST
])

# Загрузка датасетов
train_dataset_full = torchvision.datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True,
transform=transform)

# Разделение полного обучающего набора на обучающую и валидационную выборки (без scikit-learn)
dataset_size = len(train_dataset_full)
indices = list(range(dataset_size))
np.random.shuffle(indices)
split = int(0.8 * dataset_size) # 80% для обучения, 20% для валидации
```

```

train_indices, val_indices = indices[:split], indices[split:]

train_dataset = Subset(train_dataset_full, train_indices)
val_dataset = Subset(train_dataset_full, val_indices)

# DataLoader для каждой выборки
batch_size = 64
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print(f"Размер обучающей выборки: {len(train_dataset)}")
print(f"Размер валидационной выборки: {len(val_dataset)}")
print(f"Размер тестовой выборки: {len(test_dataset)}")

# 2. Визуализация примеров MNIST
def plot_mnist_examples():
    plt.figure(figsize=(10, 5))
    for i, (images, labels) in enumerate(train_loader):
        if i >= 1:
            break
        images = images.numpy()
        labels = labels.numpy()
        for j in range(10):
            plt.subplot(2, 5, j + 1)
            plt.imshow(images[j].squeeze(), cmap='gray')
            plt.title(f"Цифра: {labels[j]}")
            plt.axis('off')
    plt.tight_layout()
    plt.savefig('mnist_examples.png')
    plt.show()

print("\nВизуализация примеров MNIST...")
plot_mnist_examples()

# 3. Архитектура многослойного перцептрона
class MLP(nn.Module):

```

```

def init(self, input_size=784, hidden_sizes=[128], output_size=10, activation='relu'):
    super(MLP, self).init()
    self.activation_name = activation

    # Создание слоев
    layers = []
    prev_size = input_size

    # Скрытые слои
    for hidden_size in hidden_sizes:
        layers.append(nn.Linear(prev_size, hidden_size))
        if activation == 'relu':
            layers.append(nn.ReLU())
        elif activation == 'sigmoid':
            layers.append(nn.Sigmoid())
        prev_size = hidden_size

    # Выходной слой
    layers.append(nn.Linear(prev_size, output_size))

    self.model = nn.Sequential(*layers)

def forward(self, x):
    # Преобразование изображения в вектор (flattening)
    x = x.view(x.size(0), -1)
    return self.model(x)

# 4. Функции потерь
def mse_loss(outputs, targets):
    # Преобразование меток в one-hot кодировку
    one_hot_targets = torch.zeros(outputs.size(0), outputs.size(1)).to(outputs.device)
    one_hot_targets.scatter_(1, targets.unsqueeze(1), 1)
    # Для MSE нужно применить Softmax к выходам
    softmax_outputs = torch.softmax(outputs, dim=1)
    return nn.MSELoss()(softmax_outputs, one_hot_targets)

def cross_entropy_loss(outputs, targets):
    # В PyTorch CrossEntropyLoss объединяет LogSoftmax и NLLLoss

```

```

return nn.CrossEntropyLoss()(outputs, targets)

def kl_divergence_loss(outputs, targets):
    # Применяем LogSoftmax к выходам нейросети
    log_probs = nn.LogSoftmax(dim=1)(outputs)
    # Преобразуем метки в one-hot кодировку
    one_hot_targets = torch.zeros(outputs.size(0), outputs.size(1)).to(outputs.device)
    one_hot_targets.scatter_(1, targets.unsqueeze(1), 1)

    # Применяем Softmax к целевым значениям
    soft_targets = nn.Softmax(dim=1)(one_hot_targets * 10) # Усиливаем уверенность

    # KL Divergence Loss
    return nn.KLDivLoss(reduction='batchmean')(log_probs, soft_targets)

# 5. Обучение модели
def train_model(model, train_loader, val_loader, criterion, optimizer, num_epochs=10,
device='cpu',
                method_name='unknown'):
    model = model.to(device)
    train_losses = []
    val_losses = []
    train_accuracies = []
    val_accuracies = []

    print(f"\nНачало обучения с методом оптимизации: {method_name}")
    print("=" * 60)

    start_time = time.time()

    for epoch in range(num_epochs):
        epoch_start_time = time.time()
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for batch_idx, (inputs, targets) in enumerate(train_loader):

```

```

inputs, targets = inputs.to(device), targets.to(device)

# Для LBFGS нужен особый подход
if isinstance(optimizer, optim.LBFGS):
    def closure():
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        return loss

    loss = optimizer.step(closure)
    outputs = model(inputs)
else:
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = criterion(outputs, targets)
    loss.backward()
    optimizer.step()

# Подсчет точности
_, predicted = torch.max(outputs.data, 1)
total += targets.size(0)
correct += (predicted == targets).sum().item()
running_loss += loss.item() * inputs.size(0)

epoch_loss = running_loss / total
epoch_acc = correct / total
train_losses.append(epoch_loss)
train_accuracies.append(epoch_acc)

# Валидация
model.eval()
val_loss = 0.0
val_correct = 0
val_total = 0

with torch.no_grad():
    for inputs, targets in val_loader:
        inputs, targets = inputs.to(device), targets.to(device)

```

```

        outputs = model(inputs)

        loss = criterion(outputs, targets)
        val_loss += loss.item() * inputs.size(0)

        _, predicted = torch.max(outputs.data, 1)
        val_total += targets.size(0)
        val_correct += (predicted == targets).sum().item()

    val_epoch_loss = val_loss / val_total
    val_epoch_acc = val_correct / val_total
    val_losses.append(val_epoch_loss)
    val_accuracies.append(val_epoch_acc)

    epoch_time = time.time() - epoch_start_time
    print(f'Эпоха [{epoch + 1}/{num_epochs}], Время: {epoch_time:.2f} с')
    print(f'Обучение: Loss = {epoch_loss:.4f}, Точность = {epoch_acc:.4f}')
    print(f'Валидация: Loss = {val_epoch_loss:.4f}, Точность = {val_epoch_acc:.4f}')

    total_time = time.time() - start_time
    print("=" * 60)
    print(f'Обучение завершено. Общее время: {total_time:.2f} секунд')

    return {
        'train_losses': train_losses,
        'val_losses': val_losses,
        'train_accuracies': train_accuracies,
        'val_accuracies': val_accuracies,
        'model': model,
        'total_time': total_time
    }

```

6. Оценка на тестовых данных

```

def evaluate_model(model, test_loader, device='cpu'):
    model.eval()
    correct = 0
    total = 0
    test_loss = 0

```

```

with torch.no_grad():
    for inputs, targets in test_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        outputs = model(inputs)
        loss = cross_entropy_loss(outputs, targets)
        test_loss += loss.item() * inputs.size(0)
_, predicted = torch.max(outputs.data, 1)
total += targets.size(0)
correct += (predicted == targets).sum().item()

accuracy = correct / total
avg_loss = test_loss / total
print(f'\nТестирование:')
print(f' Loss: {avg_loss:.4f}, Точность: {accuracy:.4f} ({correct}/{total})')
return accuracy, avg_loss

```

7. Визуализация результатов обучения

```

def plot_training_results(results_dict, title, save_path=None):
    plt.figure(figsize=(12, 5))

    # График функции потерь
    plt.subplot(1, 2, 1)
    for name, results in results_dict.items():
        plt.plot(results['train_losses'], label=f'{name} (обучение)')
        plt.plot(results['val_losses'], '--', label=f'{name} (валидация)')

    plt.xlabel("Эпохи")
    plt.ylabel('Loss')
    plt.title(f'{title}: Функция потерь')
    plt.legend()
    plt.grid(True)

    # График точности
    plt.subplot(1, 2, 2)
    for name, results in results_dict.items():
        plt.plot(results['train_accuracies'], label=f'{name} (обучение)')
        plt.plot(results['val_accuracies'], '--', label=f'{name} (валидация)')

    plt.xlabel("Эпохи")

```

```
plt.ylabel('Точность')
plt.title(f'{title}: Точность')
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
if save_path:
    plt.savefig(save_path)
plt.show()
```

8. Демонстрация работы модели на тестовых примерах

```
def demonstrate_model(model, test_loader, device='cpu', num_examples=10):
```

```
    model.eval()
    examples = []
```

```
    with torch.no_grad():
```

```
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
```

```
        # Собираем примеры
```

```
        for i in range(inputs.size(0)):
            if len(examples) < num_examples:
                examples.append((inputs[i].cpu(), labels[i].item(), predicted[i].item()))
            else:
                break
        if len(examples) >= num_examples:
            break
```

```
    # Визуализация
```

```
    plt.figure(figsize=(15, 4))
    for i, (img, true_label, pred_label) in enumerate(examples):
        plt.subplot(2, 5, i + 1)
        plt.imshow(img.squeeze(), cmap='gray')
        color = 'green' if true_label == pred_label else 'red'
        plt.title(f'Истинн.: {true_label}\nПредсказ.: {pred_label}', color=color)
        plt.axis('off')
```

```

plt.tight_layout()
plt.savefig('model_predictions.png')
plt.show()

# 9. Эксперимент с различными функциями потерь
print("\n" + "=" * 80)
print("ЭКСПЕРИМЕНТ: Исследование различных функций потерь")
print("=" * 80)

# Словарь функций потерь для экспериментов
loss_functions = {
    'MSE': mse_loss,
    'CrossEntropy': cross_entropy_loss,
    'KLDivergence': kl_divergence_loss
}

# Хранение результатов
loss_results = {}

# Основные параметры для всех моделей
input_size = 784 # 28x28 пикселей
hidden_sizes = [128, 64] # Архитектура
output_size = 10 # 10 классов (цифры 0-9)
learning_rate = 0.001
num_epochs = 5 # Уменьшено для быстрого выполнения
activation = 'relu'

for loss_name, loss_fn in loss_functions.items():
    print(f"\nОбучение с функцией потерь: {loss_name}")

    # Создание модели
    model = MLP(input_size, hidden_sizes, output_size, activation)

    # Использование Adam для всех функций потерь
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    # Обучение модели
    results = train_model(model, train_loader, val_loader, loss_fn, optimizer,
                           num_epochs=num_epochs, device=device, method_name=f'Adam + {loss_name}')

```

```

# Оценка на тестовых данных
print(f"\nОценка модели с функцией потерь {loss_name} на тестовых данных:")
test_acc, test_loss = evaluate_model(results['model'], test_loader, device)
results['test_accuracy'] = test_acc
results['test_loss'] = test_loss

loss_results[loss_name] = results

# Сохранение модели
os.makedirs('models', exist_ok=True)
torch.save(model.state_dict(), f'models/model_loss_{loss_name}.pth')
# Визуализация результатов для функций потерь
plot_training_results(loss_results, 'Различные функции потерь', 'loss_comparison.png')

# 10. Демонстрация работы лучшей модели
best_model = loss_results['CrossEntropy']['model'] # CrossEntropy обычно работает лучше всего
print("\nДемонстрация работы лучшей модели (CrossEntropy) на тестовых примерах:")
demonstrate_model(best_model, test_loader, device)

# 11. Вывод весов лучшей модели
print(f"\n" + "=" * 80)
print(f"ВЕСА ЛУЧШЕЙ МОДЕЛИ (CrossEntropy)")
print("=" * 80)

# Вывод весов первого слоя
print("\nСинаптические веса первого слоя:")
first_layer = best_model.model[0] # Первый линейный слой

if isinstance(first_layer, nn.Linear):
    weights = first_layer.weight.data.cpu().numpy()
    bias = first_layer.bias.data.cpu().numpy()

    print(f"Форма весов: {weights.shape}")
    print(f"Форма смещения (bias): {bias.shape}")

# Вывод фрагмента весов для отчета
print("\nФрагмент весов первого слоя (первые 5 нейронов, первые 10 входов):")
for i in range(min(5, weights.shape[0])):
    print(f"Нейрон {i}: {weights[i, :10]}")

```

```

# Визуализация распределения весов
plt.figure(figsize=(10, 6))
plt.hist(weights.flatten(), bins=50, alpha=0.7)
plt.title('Распределение весов первого слоя')
plt.xlabel('Значение веса')
plt.ylabel('Частота')
plt.grid(True, alpha=0.3)
plt.savefig('weights_distribution.png')
plt.show()

# Визуализация весов как изображений
plt.figure(figsize=(12, 8))
for i in range(min(16, weights.shape[0])):
    plt.subplot(4, 4, i + 1)
    weight_image = weights[i].reshape(28, 28)
    plt.imshow(weight_image, cmap='hot')
    plt.title(f'Нейрон {i}')
    plt.axis('off')
plt.tight_layout()
plt.savefig('weights_as_images.png')
plt.show()

else:
    print("Первый слой не является линейным. Веса не могут быть извлечены стандартным
способом.")

# 12. Финальные результаты
print("\n" + "=" * 80)
print("РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ")
print("=" * 80)

print("\nСравнение функций потерь:")
for loss_name, results in loss_results.items():
    print(
        f"    {loss_name}: Точность на тесте = {results['test_accuracy']:.4f}, Время обучения =
{results['total_time']:.2f} сек")

print("\nВсе результаты успешно сохранены!")
print("Графики сохранены в файлах:")
print("- mnist_examples.png")

```

```
print("- loss_comparison.png")  
print("- model_predictions.png")  
print("- weights_distribution.png")  
print("- weights_as_images.png")  
print("\nМодели сохранены в папке 'models/'.")
```